Bugfix Log

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

Per CONST-MD-Bugfix-Documentation, every bug surfaced during implementation gets a permanent entry below: title, root cause, affected files, fix description, regression guard.

Entries are append-only; do not edit historical entries except to add clarification footnotes.

2026-04-27 â€” boba-jackett implementation (plan 2026-04-27-jackett-management-ui-and-system-db.md)

1. Master-key two-step write silent data-loss window

Severity: HIGH (silent credential loss possible on crash mid-bootstrap).

Root cause: Initial implementation of `bootstrap.EnsureMasterKey` generated `BOBA_MASTER_KEY` in memory, returned it to the caller, and *then* wrote `.env` in a separate step. If the process crashed, was killed, or the host lost power between those two operations, the caller would proceed to encrypt credentials with a key that disk had no record of â€” making them permanently undecipherable.

Affected files: - `qBitTorrent-go/internal/bootstrap/bootstrap.go` - `qBitTorrent-go/internal/envfile/write.go`

Fix: Collapse the generate + write into a single `envfile.Atomic` write (`tmpfile + fsync + rename`) before the function returns. Caller never sees a key that hasnâ€™t been durably persisted. Commit `c0f20bc`.

Regression guard: - `qBitTorrent-go/internal/bootstrap/bootstrap_test.go::TestEnsureMasterKeyDoesNotDuplicateHeader` - `qBitTorrent-go/internal/envfile/write_test.go` atomic-write test layer

---

2. SQLite database file mode `0644` (world-readable credentials)

Severity: CRITICAL (any local user could read encrypted credentials + ciphertext-only attack surface).

Root cause: `modernc.org/sqlite` creates database files with the processâ€™s default umask, which on the container yielded `0644` (world-readable). The Go serviceâ€™s `.env` was already `0600` via `envfile.Atomic`, but `boba.db` was leaking â€” a defense-in-depth breach since the rows are encrypted, but still a violation of the principle â€œthe DB file should be readable only by the service ownerâ€.

Affected files: - `qBitTorrent-go/internal/db/conn.go`

Fix: After the first `Open()` succeeds, `db.Open` calls `os.Chmod(path, 0o600)` (and the `-wal` / `-shm` siblings if present), collapsing the file mode to owner-only. Commit `8405167`.

Regression guard: - Layer 4 security challenge `challenges/scripts/boba_db_file_perms_challenge.sh` - `qBitTorrent-go/internal/db/credential_leak_test.go`

---

3. `AutoconfigResult` nil slices marshalled as JSON `null`

Severity: MEDIUM (silent client-breaking schema drift; UI rendered â€œundefinedâ€ badges instead of â€œ0 itemsâ€).

Root cause: Goâ€™s `json.Marshal` serializes a nil slice as `null`, not `[]`. When `Autoconfigure()` ran on a fresh DB with no discovered credentials, the `discovered_credentials`, `configured_now`, and `already_present` fields all marshalled as `null`. The OpenAPI 3.1 schema declared them as `array`, so clients (including the Angular dashboard) treated `null` as a contract violation.

Affected files: - `qBitTorrent-go/internal/jackett/autoconfig.go`

Fix: Pre-allocate empty slices (`[]string{}`) at the top of `Autoconfigure()` so a â€œnothing to doâ€ run still serializes as `{...,"discovered":[],"configured_now":[],...}`. Commit `6f3dbaf`.

Regression guard: - Layer 6 contract test `qBitTorrent-go/tests/contract/openapi_test.go` (validates each named field is `array` per OpenAPI schema even on empty runs).

---

4. Catalog `ReplaceAll` empty-input wipe risk

Severity: HIGH (would permanently wipe the indexer catalog if Jackett returned an empty response; UI would show no indexers and fuzzy matcher would have nothing to match against).

Root cause: `repos.IndexerCatalog.ReplaceAll(rows []Row)` unconditionally `DELETE`d existing rows then `INSERT`ed the new ones. If `rows` was empty (Jackett momentarily unhealthy, transient 502, or catalog parse failure), the delete still happened, leaving an empty catalog table. Next refresh would have nothing to compare against.

Affected files: - `qBitTorrent-go/internal/db/catalog.go` (or equivalent repo file)

Fix: Refuse the operation early with `errors.New("repos: ReplaceAll refusing empty replacement")`. The caller (autoconfig orchestrator) treats this as a soft error, logs it, and leaves the previous catalog intact. Commit `8d71df1`.

Regression guard: - `qBitTorrent-go/internal/jackettapi/catalog_test.go::TestRefreshCatalogAllTemplatesFailReturns200WithErrors`

---

5. `TestSearchHandler_QueueFull` pre-existing flake (NOT a boba-jackett bug)

Severity: LOW (test-only; flagged for follow-up).

Status: NOT FIXED in this plan. Confirmed unrelated to boba-jackett work.

Location: `qBitTorrent-go/internal/api/api_test.go`

Symptom: Occasional intermittent failure under load when the search queue fills before the testâ€™s `wait` returns.

Action: Logged here for traceability so future audits donâ€™t mis-attribute it. Open issue suggestion: stabilise by injecting a deterministic queue-full hook instead of racing against real goroutine scheduling.